Literature Overview 🔍

Table of Contents

- Multi-Agent System (MAS) and SLMs
  - MAS Literature
  - Small Agents Literature
- Datasets
- Fine-tuning
- Thesis relevance
- Example RQs

Multi-Agent System (MAS) and SLMs

MAS Literature

Small Language Models are the Future of Agentic AI (2506.02153v2)

Belcak et al. argue that the current reliance on generalist LLMs in agentic architectures represents an economically inefficient misallocation of computational resources.

- They advocate for heterogeneous agentic systems where complex goals are decomposed into modular sub-tasks.
- The authors observe that agentic applications typically act as heavily instructed gateways that restrict a language model to a very small subset of its overall capabilities.
- Because tasks are broken down into narrow, scoped operations, the broad "semantic hub" and generalized reasoning capabilities of massive LLMs become unnecessary at the execution level.
- By combining the precision and efficiency of SLMs with the selective use of LLMs for general reasoning, developers can build modular, "Lego-like" systems that are easier to debug, adapt, and scale.

> 📝 Small Language Models are the Future of Agentic AI (2506.02153v2), p.5

> The above-mentioned “Lego-like” composition of agentic intelligence—scaling out by adding small, specialized experts instead of scaling up monolithic models—yields systems that are cheaper, faster to debug, easier to deploy, and better aligned with the operational diversity of real-world agents. When combined with tool calling, caching, and fine-grained routing, SLM-first architectures appear to offer the best path forward for cost-effective, modular, and sustainable agentic AI

Agentic Reasoning A Streamlined Framework for Enhancing LLM Reasoning with Agentic Tools

- Wu et al. propose *Agentic Reasoning*, a framework designed to enhance large language model reasoning by integrating external, tool-using agents.
- This approach decouples **high-level reasoning from execution by allowing the main reasoning LLM to delegate specific sub-tasks to specialized auxiliary agents, which prevents the main model from being disrupted by tasks like writing and executing code.**
- The authors experimentally identify three universally effective agentic tools: a Web-Search agent, a Coding agent, and a Mind-Map memory agent.
- A key contribution relevant to minimal orchestration is their finding on **tool quantity versus quality; ablation studies revealed that adding massive toolboxes (such as LangChain's 109 tools) actually degraded overall performance due to an increased risk of inappropriate tool selection.**
- The framework introduces the Mind-Map agent, which dynamically constructs a structured knowledge graph to store reasoning context, track logical relationships, and maintain coherence over long, multi-step reasoning chains.
- Furthermore, the authors demonstrate that wrapping tools in an "agentic" layer (where the tool is managed by an LLM) significantly reduces errors compared to traditional direct API calling, as the agent can self-monitor, express uncertainty, and prevent cascading failures.

Agentic Reasoning A Streamlined Framework for Enhancing LLM Reasoning with Agentic Tools (2502.04644v2), p.15

> Using agentic tool calls rather than direct API calls offers several advantages: 1. Overcoming single-model token limitations: By structuring the workflow agentically, the system can break free from the token generation limits of a single LLM. This enables it to produce longer, higher-quality reasoning chains than would be possible within the token budget of a single model call. 2. Managing uncertainty and reducing error propagation: Agents can self-monitor and assign varying levels of confidence to their

> outputs. As noted earlier, this mechanism helps the reasoning model treat low-confidence outputs as tentative, thereb

Multi-agent Architecture Search via Agentic Supernet

- A framework that moves away from designing monolithic, static agentic systems by optimizing an **agentic supernet** - a probabilistic and continuous distribution of multi-agent architectures.
- The authors argue that contemporary automated methods usually seek a complex, one-size-fits-all system, which fails to dynamically allocate inference resources based on the specific difficulty and domain of each query. To solve this, MaAS samples query-dependent architectures from the supernet, tailoring the system's resource allocation, including LLM calls and tool usage, to the task at hand.
- A critical component of their architecture is the "early-exit operator," which allows the system to halt multi-agent sampling early for simple queries, preventing unnecessary redundancy and resource expenditure.
- **The paper provides a strong counterpoint to static orchestration, showing that different domains require fundamentally different agentic topologies to succeed efficiently**

> Multi-agent Architecture Search via Agentic Supernet (2502.04180v2), p.2
>
> > Figure 1. (Left) The building blocks of MaAS; (Right) When confronting different queries, the agentic supernet adaptively samples tailored multi-agent architecture in a query-dependent manner

In-the-Flow Agentic System Optimization for Effective Planning and Tool Use (2510.05592v1)

See link: https://agentflow.stanford.edu/
Li et al. introduce AGENTFLOW, an agentic framework designed to overcome the limitations of monolithic tool-integrated reasoning models.

- The authors argue that monolithic models, which train a single policy to interleave reasoning and tool calls under full context, suffer from instability and brittle generalization as task horizons and tool diversity increase.
- AGENTFLOW decomposes complex work across four specialized modules: an **Action Planner, a Tool Executor, an Execution Verifier, and a Solution Generator.**

- These modules coordinate iteratively via a shared, evolving memory that creates a deterministic and structured record of the reasoning process, which bounds context growth and enables transparent state tracking.
- By orchestrating tasks through this modular system, the authors demonstrate that an **agentic framework powered by a 7B-parameter backbone can outperform a ~200B-parameter monolithic model (GPT-4o) across search, mathematical, and agentic domains.**

Multi-Agent Collaboration Mechanisms A Survey of LLMs (2501.06322v1)

SURVEY: Tran et al. introduce a comprehensive framework that characterizes MAS collaboration based on key dimensions: actors, types (cooperation, competition, coopetition), structures (centralized, distributed, hierarchical), and strategies (rule-based, role-based, model-based).

- The survey formally defines the **"Centralized Structure" (or star structure), where a central agent acts as a hub to manage, control, and coordinate interactions among all other participating agents.** This directly maps to our proposed Orchestrator architecture.
- The authors highlight "Role-based Protocols," noting that leveraging distinct predefined roles allows each agent to operate on segmented objectives based on their specific domain knowledge, which supports the overarching system goal.
- Importantly, **the survey warns that multi-agent systems with suboptimally designed collaboration channels can actually be outperformed by single-agent counterparts with strong prompts.** This underscores that effective orchestration and channel design are critical to MAS success.

Multi-Agent Collaboration Mechanisms A Survey of LLMs (2501.06322v1), p.20

> The rise of LLM-based multi-agent collaborative systems has been driven by the introduction of LLMs and their effectiveness as central processing brains. Inspired by human collaboration, these systems typically break complex tasks into subtasks, with agents assigned specific roles (e.g., software engineer) to focus on subtasks relevant to their expertise. Collaboration channels are critical in enabling agents to work together, facilitating capabilities such as planning and coordination. These channels are characterized by their actors (agents involved), type (e.g., cooperation, competition, or coopetition), structure (e.g., peer-to-peer, centralized, or distributed), and strategy (e.g., role-based, rule-based, or model-based)

[NOTE] nice for thesis intro!

Multi-Agent Collaboration Mechanisms A Survey of LLMs (2501.06322v1), p.21

Effective Collaboration Channels: establishing robust collaboration channels among agents is crucial for seamless collaboration. Clear protocols prevent misunderstandings and ensure efficient information exchange. As shown in AutoGen framework [134] MASs can outperform single-agent systems with effectively designed collaboration mechanisms. On the other hand, as studied in [128] MAS approach with suboptimal design for their competitive collaboration channels can be overtaken by single-agent counterpart with strong prompts.

ToolOrchestra Elevating Intelligence via Efficient Model and Tool Orchestration (2511.21689v1)

- The authors challenge the monolithic model approach by introducing the "orchestration paradigm," where intelligence emerges from a composite system managed by a central orchestrator.
- They demonstrate that simply prompting off-the-shelf LLMs to act as orchestrators is brittle and introduces systemic biases. Specifically, prompted LLMs exhibit a "self-enhancement bias" (e.g., Qwen3-8B disproportionately delegates tasks to GPT-5) or default to the strongest available tool regardless of cost (e.g., GPT-5 almost exclusively calling GPT-5-mini).
- By explicitly training a dedicated orchestrator using reinforcement learning, the system learns to **dynamically balance performance, cost, and user preferences,** proving that proper orchestration requires dedicated training rather than static heuristics.

Multi-modal Agent Tuning Building a VLM-Driven Agent for Efficient Tool Usage (2412.15606v2)

Multi-modal Agent Tuning, a framework for improving tool usage in agentic systems by fine-tuning a vision-language model (VLM) as the controller of a tool-using agent.
The authors argue that prompt-engineered agents struggle with real-world tasks because their reasoning is based only on textual input and fixed in-context examples, which limits the ability to select appropriate tools in multi-modal scenarios. To address this limitation, the paper introduces a data synthesis pipeline that automatically generates large-scale multi-modal tasks with associated tool-usage trajectories. The pipeline includes three stages:

1. Query generation
2. File generation (e.g., images or documents)
3. Trajectory generation using a zero-shot agent

These trajectories include intermediate reasoning steps, tool calls, and observations. Low-quality samples are filtered using verification modules to ensure dataset reliability.
Using the resulting **MM-Traj dataset (20K tasks),** the authors train a VLM controller that produces reasoning steps and executable tool-calling code within a ReAct-style loop.

Their resulting system, T3-Agent, demonstrates significant improvements on multi-modal benchmarks such as **GAIA and GTA**, outperforming untuned VLM agents by around 20% in accuracy.

This work highlights the importance of training agents specifically for tool-usage reasoning rather than relying solely on prompt-based orchestration, reinforcing the idea that agentic capability can be substantially improved through targeted training.
→ **Training agents on tool-use trajectories significantly improves reasoning and tool selection.**

Small Agents Literature

Small Language Models are the Future of Agentic AI (2506.02153v2)

This position paper formally contends that SLMs are sufficiently powerful, inherently more operationally suitable, and necessarily more economical for the vast majority of agentic invocations.

- The authors argue that the majority of subtasks in deployed systems are repetitive, scoped, and non-conversational, which perfectly suits the predictability of SLMs.
- Serving a 7B parameter SLM is estimated to be 10 to 30 times cheaper in terms of latency, energy consumption, and FLOPs compared to a 70B-175B parameter LLM.
- **SLMs also enforce closer behavioral alignment; they are less prone to hallucinatory format deviations when generating tool calls or interacting directly with code schemas .**
- In three case studies estimating LLM-to-SLM replacement in popular open-source agents (MetaGPT, Open Operator, Cradle), the authors estimate that 40% to 70% of LLM queries could be reliably replaced by appropriately specialized SLMs.

Agentic Reasoning A Streamlined Framework for Enhancing LLM Reasoning with Agentic Tools

While the primary experiments in this study utilize frontier models like DeepSeek-R1 and DeepSeek-V3, the architectural design strongly supports the principles of Small Language Model (SLM) ensembles. The authors highlight the advantage of **Task-specific model**

modularity, which allows different language models to be assigned to different tasks based on their strengths. Specifically, they note that an agentic setup allows for the routing of simpler subtasks—such as summarizing web search results—to lightweight, non-reasoning models. This modularity optimizes resource usage and preserves efficiency by matching the best, potentially smaller, model to each specific subtask rather than relying on a monolithic model for every step.

Multi-agent Architecture Search via Agentic Supernet

- While MaAS evaluates models of varying sizes, its core philosophy aligns with the goals of Small Language Model ensembles: **maximizing performance while strictly minimizing inference costs and token consumption.**
- The authors demonstrate that by dynamically sampling customized systems, MaAS requires only 6% to 45% of the inference costs compared to existing handcrafted or automated multi-agent systems.
- By proving that complex, token-heavy systems are largely unnecessary for simpler tasks (which can be resolved with single zero-shot I/O), MaAS reinforces the viability of utilizing smaller, specialized configurations to handle varying workloads efficiently

In-the-Flow Agentic System Optimization for Effective Planning and Tool Use (2510.05592v1)

→ Strong evidence that SLM ensembles can exceed the capabilities of frontier large models when given appropriate roles, coordination mechanisms and fine-tuning

- The entire AGENTFLOW system was instantiated using a 7B-parameter backbone (Qwen2.5-7B-Instruct) for all four of its active modules.
- Despite its small size, this SLM-based system achieved average accuracy gains of 14.9% on knowledge-intensive search and 14.5% on mathematical reasoning compared to top-performing specialized baselines.
- The authors conducted an ablation study substituting the 7B planner with a frozen GPT-4o model, finding only modest gains; this indicates that dynamic, in-the-flow co-adaptation of small agents is a more effective strategy than relying on static, large-capacity models.

Multi-Agent Collaboration Mechanisms A Survey of LLMs (2501.06322v1)

While focusing broadly on LLMs, the survey supports the SLM philosophy by noting that MAS systems distribute tasks among agents, allowing them to retain and **share diverse knowledge bases without overloading a single system.** The authors point out that role-based techniques encourage modularity and increase the reusability of individual modules, enhancing the overall system performance. **This validates the approach of using smaller, specialized experts rather than forcing a large model to handle every subtask.**

Scalability and resource maintenance are identified as open challenges
→ **good to include as motivation in the thesis**

Multi-Agent Collaboration Mechanisms A Survey of LLMs (2501.06322v1), p.28

> Scalability and Resource Maintainance. Increasing agent population poses a significant challenge in MASs. Managing resources (memory, processing time), coordination and collaboration channels among a growing number of agents introduces additional complexities, such as maintaining efficiency in agent interactions and preventing bottlenecks. Understanding the scaling laws of the behavior and performance of MASs is critical for designing architectures capable of handling large-scale collaboration.

ToolOrchestra Elevating Intelligence via Efficient Model and Tool Orchestration (2511.21689v1)

- The paper strongly reinforces the viability of SLMs, hypothesizing that small language models are entirely sufficient to serve as the "brain" if they are taught to coordinate more intelligent tools strategically.

ToolOrchestra Elevating Intelligence via Efficient Model and Tool Orchestration (2511.21689v1), p.2

> ToolOrchestra (shown in Figure 2), a novel method for training a small language model to act as the orchestrator – the “brain” of a heterogeneous tool-use agent

So, on the one hand this paper suggests the “brain” can be small and on the other hand, other papers (SLMs are the future…) suggest that once the task can be decomposed into smaller subtasks, using specialized SLMs → better (TBC)

- The authors produced Orchestrator-8B, an 8B-parameter model that achieved higher accuracy at a lower cost than previous monolithic tool-use agents.
- Instead of excessively invoking strong, expensive models for every step, Orchestrator-8B learns to make balanced, strategic tool calls, avoiding the biases shown by large foundation models.

Distilling LLM Agent into Small Models with Retrieval and Code Tools (2505.17612v2)

Kang et al. introduce Agent Distillation, a framework designed to transfer not just reasoning capabilities, but full task-solving behaviors from large LLM agents to SLMs.

- The authors argue that traditional Chain-of-Thought (CoT) distillation often fails in small models because it relies on the model to internalize rare factual knowledge or perform precise computations, leading to hallucinations.
- Instead of static reasoning, Agent Distillation teaches the SLM to navigate "reason-act-observe" trajectories. This approach enables the SLM to actively use external tools-specifically retrieval and code execution-to solve problems adaptively.
- The results strongly validate the efficacy of small models: SLMs with as few as 0.5B, 1.5B, and 3B parameters achieved performance competitive with next-tier larger models (1.5B, 3B, 7B) that were fine-tuned using standard CoT distillation.

Multi-modal Agent Tuning Building a VLM-Driven Agent for Efficient Tool Usage (2412.15606v2)

Although the paper focuses on multi-modal agents, it provides indirect support for the Small Language Model (SLM) paradigm → the authors demonstrate that relatively small vision-language models (e.g., Qwen2-VL-7B and MiniCPM-V-8.5B) can serve as effective controllers for complex multi-step tool-using agents when trained on high-quality trajectory data.
After trajectory tuning, these relatively small models significantly improve their tool-selection and reasoning capabilities, narrowing the gap with larger proprietary systems.

This suggests that model scale alone is not sufficient for agentic reasoning, and that training for tool interaction and structured reasoning trajectories can substantially improve the effectiveness of smaller models. **The work therefore supports the broader argument that specialized training and architectural design can allow smaller models to perform complex agentic tasks efficiently, aligning with the emerging trend toward modular agentic systems composed of lightweight models.**

Self-Hinting Language Models Enhance Reinforcement Learning (2602.03143v1)

The paper demonstrates a strategy for improving reasoning capability without scaling model size. Instead of relying on larger models or external teacher models, the authors show that smaller models can improve reasoning performance through self-generated intermediate supervision.

The method includes: online hint generation, adaptive hint scheduling, privileged supervision during training

Hints are only used during training and removed at inference time. This allows smaller models to learn improved reasoning policies without requiring additional inference components. Empirical results show improvements across several model sizes, suggesting that structured intermediate reasoning signals can boost smaller models' learning efficiency.

Datasets

Agentic Reasoning A Streamlined Framework for Enhancing LLM Reasoning with Agentic Tools

- The authors utilize the **GAIA benchmark** to evaluate their system's ability to handle reasoning, web browsing, and tool-use proficiency. Their results on GAIA validate the necessity of multi-agent collaboration for complex tasks; they note that GAIA requires a combination of advanced reasoning and tool-use that benefits heavily from their agentic framework, surpassing prior search-in-reasoning approaches.
- The paper also evaluates expert-level problem-solving using datasets like **GPQA and Humanity's Last Exam,** demonstrating that well-orchestrated agentic systems can close the gap with proprietary, closed-source models on highly difficult, multi-step benchmarks.

Multi-agent Architecture Search via Agentic Supernet

- MaAS conducts comprehensive evaluations across several benchmarks, **featuring GAIA for diverse tool usage, as well as HumanEval and MBPP for code generation, and GSM8K and MATH for mathematical reasoning.**

Multi-agent Architecture Search via Agentic Supernet (2502.04180v2), p.5

> Tasks and Benchmarks. We evaluate MaAS on six public benchmarks covering three domains: (1) math reasoning, GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), and MultiArith (Roy & Roth, 2016); (2) code generation, HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021)); and (3) tool use, GAIA (Mialon et al., 2023).

The authors specifically note that in the GAIA benchmark, no single static system is optimal for both file reading and web searching tasks simultaneously.

Multi-agent Architecture Search via Agentic Supernet (2502.04180v2), p.2

> there is no single system that is optimal for both file reading and web searching tasks, leaving practitioners with no alternative but to split the benchmark and optimize separately (Zhuge et al., 2024).

By adaptively sampling customized agentic systems based on the specific domain of the GAIA task, MaAS achieved substantial improvements over baselines, securing an 18.38% improvement on Level 1 tasks and a 17.61% improvement on Level 2 tasks.

In-the-Flow Agentic System Optimization for Effective Planning and Tool Use (2510.05592v1)

The researchers evaluated AGENTFLOW across ten benchmarks spanning **knowledge-intensive search (HotpotQA, 2Wiki, Musique, Bamboogle), agentic reasoning (GAIA), mathematical reasoning (AIME24, AMC23, GameOf24), and scientific reasoning (GPQA, MedQA)**

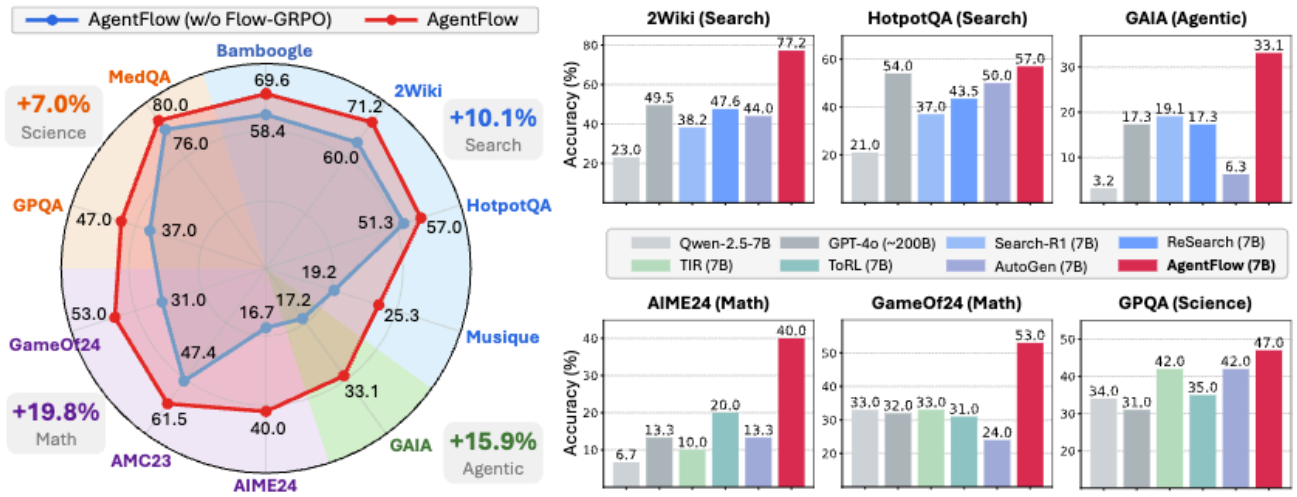

Figure 1: **Left:** Performance of AGENTFLOW with a 7B-scale backbone before and after Flow-GRPO tuning across ten diverse reasoning benchmarks. Flow-GRPO substantially improves performance by enhancing planning quality and tool-calling reliability. **Right:** AGENTFLOW achieves consistent gains over top baselines, including base LLMs, tool-integrated RL models, and training-free agentic systems. All 7B results use Qwen2.5-7B-Base/Instruct as the backbone and tools.

Multi-Agent Collaboration Mechanisms A Survey of LLMs (2501.06322v1)

The survey observes that current benchmarks for LLM-based multi-agent collaborative systems primarily focus on metrics such as success rate, task outcomes, cost-effectiveness, and collaborative efficiency - the authors emphasize **the need for fine-grained evaluation at both the individual agent and the collaboration levels to enable root cause analysis, offering insights into individual behaviors and overall system dynamics. → good to add as thesis motivation**

ToolOrchestra Elevating Intelligence via Efficient Model and Tool Orchestration (2511.21689v1)

The paper evaluates multi-agent orchestration on highly complex reasoning benchmarks: **Humanity's Last Exam (HLE), FRAMES, and τ^2-Bench.** Because verifiable multi-turn tool-use data is scarce, the authors **developed an automatic data synthesis pipeline to create a**

dataset called ToolScale. ToolScale simulates rich user-agent-tool environments across 10 domains, generating diverse tasks with corresponding ground truth solutions to enable end-to-end RL training.

ToolOrchestra Elevating Intelligence via Efficient Model and Tool Orchestration (2511.21689v1), p.15

> B. Evaluation Benchmarks · **Humanity's Last Exam (HLE)** [1]. A large-scale benchmark comprising PhD-level questions across mathematics, humanities, natural sciences and more. It evaluates the model capabilities to perform iterative search and intensive reasoning. Questions are multiple-choice or short-answer, with 10–14% requiring images. We use the text-only subset, designed to be unambiguous and not solvable by simple web search. · **FRAMES** [13]. A dataset for end-to-end evaluation of retrieval-augmented generation (RAG), covering factuality, retrieval accuracy, and reasoning. It contains 824 multi-hop questions requiring 2–15 Wikipedia articles, spanning numerical, tabular, temporal, and multi-constraint reasoning. · τ **2-Bench** [12]. A benchmark to evaluate model capabilities to use tools and solve problems in conversations with users. It includes tasks in three domains: telecom, retail and airline.

Distilling LLM Agent into Small Models with Retrieval and Code Tools (2505.17612v2)

The authors evaluated their framework across a broad spectrum of benchmarks, specifically factual and mathematical domains. For factual reasoning (requiring the retrieval tool), they utilized HotpotQA, Bamboogle, MuSiQue, and 2WikiMultiHopQA. For mathematical reasoning (requiring the code execution tool), they evaluated on MATH, GSM-Hard, AIME, and OlymMath. **To train the distilled agents, they used a subset of 1,000 HotpotQA examples and 2,000 MATH examples to generate the teacher trajectories.**

Distilling LLM Agent into Small Models with Retrieval and Code Tools (2505.17612v2), p.1

> Performance comparison of different sizes of Qwen2.5-Instruct models [1] on the average accuracy of four factual reasoning tasks (HotpotQA [2], Bamboogle [3], MuSiQue [4], 2WikiMultiHopQA [5]) and four mathematical reasoning tasks (MATH [6], GSM-Hard [7], AIME [8], OlymMATH [9]). Distillation is done using the 32B model as the teacher and models ranging from 0.5B to 7B as student

Multi-modal Agent Tuning Building a VLM-Driven Agent for Efficient Tool Usage (2412.15606v2)

MM-Traj, a synthetic dataset designed specifically for training multi-modal agents to use tools. The dataset contains: 20K tasks; 15K associated files; diverse modalities including images, PDFs, spreadsheets, and audio files; multi-step reasoning trajectories involving tool usage. **Tasks typically require 2–6 tool interactions, reflecting realistic agentic workflows.**

The authors evaluate their trained agents on the GTA and GAIA benchmarks, which assess multi-modal reasoning, tool usage, and long-horizon task solving.

Fine-tuning

Small Language Models are the Future of Agentic AI (2506.02153v2)

Belcak et al. identify agentic interactions as natural pathways for gathering highly specialized data to iteratively improve SLMs.

- Because agentic sub-tasks demand strict adherence to output formatting (e.g., JSON or Python for tool calls), SLMs can be easily fine-tuned to enforce a single formatting decision, eliminating the overhead required for an LLM to juggle multiple formats.
- The authors outline a continuous **LLM-to-SLM Agent Conversion Algorithm** where initial deployments log prompts and tool calls to build curated datasets. Unsupervised clustering is then used to identify recurring task patterns, which are subsequently used to train specialized SLMs using Parameter-Efficient Fine-Tuning (PEFT) techniques like LoRA or QLoRA . Because SLM fine-tuning requires only a few GPU-hours, agentic behaviors can be rapidly specialized, adapted, or corrected overnight.

Multi-agent Architecture Search via Agentic Supernet

- Instead of traditional parameter fine-tuning, Zhang et al. utilize an optimization strategy based on environmental feedback to jointly update the probability distribution of agentic operators and the operators themselves.
- Because agentic operators include black-box tool usage and natural language prompts that cannot be optimized via numerical backpropagation, the authors employ **agent-based textual gradients**. These textual gradients are agent-generated analyses that provide structural updates, prompt refinements, and model temperature adjustments to self-evolve the multi-agent system.

Multi-agent Architecture Search via Agentic Supernet (2502.04180v2), p.5

> Figure 3. The demonstration of textual gradient.

- **Cross-Model Transferability of the Tuned Architecture:** The probabilistic routing distribution, once fine-tuned via this cost-constrained optimization, does not overfit to a single model. The authors showed that a supernet optimized using a specific LLM (like GPT-4o-mini) successfully transferred to other backbones (like Qwen-2.5-72b and Llama-3.1-70b), yielding performance improvements of approximately 5%, proving that the optimized orchestration strategy generalizes beyond its specific training environment.
- The optimization of the agentic supernet demonstrates **strong inductive capabilities, meaning the fine-tuned controller can generalize to operators it was never explicitly trained on.** For instance, when the researchers held out the "Debate" operator during training and introduced it only at inference, the tuned system was still able to reasonably activate and utilize it at appropriate proportions.

In-the-Flow Agentic System Optimization for Effective Planning and Tool Use (2510.05592v1)

The challenge of long-horizon credit assignment in agentic systems is tackled by by introducing Flow-based Group Refined Policy Optimization (Flow-GRPO).

- Unlike traditional offline Supervised Fine-Tuning (SFT), which is decoupled from live system dynamics, Flow-GRPO optimizes the planner on-policy inside the multi-turn loop. **The authors demonstrate that offline SFT actually leads to a catastrophic performance collapse** (a 19.0% accuracy drop), as token-level imitation fails to teach the planner how to adapt to dynamic tool feedback or recover from compounding errors.
- Flow-GRPO solves the sparse-reward problem by broadcasting a single, verifiable final-outcome reward to the entire trajectory, aligning every local planner decision with global success.This in-the-flow reinforcement learning significantly reduces tool-calling error rates and teaches the planner to autonomously discover effective solution pathways and adapt its tool preferences based on the domain.
- During the reinforcement learning fine-tuning phase, the planner is trained using a mixture of paired question-answer examples from the Search-R1 and DeepMath datasets to cover both knowledge-intensive search and mathematical domains. This fine-tuning process proves highly effective and adaptable, offering consistent performance gains across tasks as the backbone LLM scales in size, such as from 3B to 7B parameters

ToolOrchestra Elevating Intelligence via Efficient Model and Tool Orchestration (2511.21689v1)

- The Orchestrator is fine-tuned using **Group Relative Policy Optimization (GRPO),** moving beyond standard supervised fine-tuning. **The RL reward design is multi-objective, calculating rewards based on three factors: the correctness of the final outcome,**

efficiency (penalizing the monetary cost and wall-clock latency of the trajectory), and adherence to user tool preferences.

- To stabilize this complex agentic RL training, the authors applied specific techniques during backward propagation, such as homogeneity filtering (ignoring batches with low reward variance) and format consistency filtering.

Distilling LLM Agent into Small Models with Retrieval and Code Tools (2505.17612v2)

The paper outlines highly specific fine-tuning and inference techniques necessary to make small agents reliable tool-users, rather than just text generators.

- Parameter-Efficient Tuning: The student models were fine-tuned using **LoRA on all linear layers**, demonstrating that full fine-tuning is not strictly necessary to impart complex agentic behaviors.
- **First-Thought Prefix (FTP):** To generate high-quality training data, the authors found that simply prompting a teacher LLM to act as an agent sometimes degraded its reasoning. They solved this by prefixing the teacher's trajectory with the first step of a standard CoT prompt, which forced the teacher into a correct reasoning path before it generated actions for the student to learn from.
- **Self-Consistent Action Generation (SAG):** To address the fragility of SLMs at inference time, the authors implemented SAG to improve test-time robustness. Because small models often produce misformatted code or invalid actions, SAG samples multiple action sequences and uses a lightweight interpreter to filter out execution errors. The system then performs majority voting on the resulting valid observations to select the most consistent outcome, reducing code execution and parsing errors.

Multi-modal Agent Tuning Building a VLM-Driven Agent for Efficient Tool Usage (2412.15606v2)

Trajectory Tuning for Tool Usage, a fine-tuning strategy where a VLM is trained to reproduce tool-usage trajectories consisting of reasoning steps and executable code. **Training is performed using the MM-Traj dataset and optimizes the probability of generating the correct reasoning and tool-calling sequence at each step.**

The authors use LoRA-based parameter-efficient fine-tuning, freezing the vision encoder and updating only the language model components of the VLM. Interestingly, the model is not trained to predict the final answer directly, encouraging it instead to rely on external tools for solving tasks.

This training approach improves: tool selection accuracy, reasoning coherence, executable code generation. **These results suggest that trajectory-level training data is crucial for**

developing reliable tool-using agents, which aligns with broader trends in agent training research

[Self-Hinting Language Models Enhance Reinforcement Learning (2602.03143v1)

The training procedure builds on **Group Relative Policy Optimization (GRPO)**, a reinforcement learning algorithm commonly used for language model alignment. The key issue addressed in the paper is advantage collapse. When all generated responses in a batch receive the same reward (e.g., all zero), the policy gradient becomes zero and learning stalls.

The proposed **SAGE framework** modifies the training pipeline as follows:

1. For each input prompt, the model generates a **hint**.
2. The hint is appended to the prompt.
3. The model generates a **solution conditioned on the hint**.
4. The solution is evaluated with the reward function.

Hints introduce **diversity in reasoning trajectories**, increasing the probability of at least one correct solution in the rollout group. The paper also introduces **adaptive hint scheduling**. Hints are only applied when reward collapse is detected. If all trajectories in a batch receive identical rewards, the system increases the hint strength to guide exploration. Additionally, the paper proposes **self-hinting**, where the model periodically generates hints itself during training rather than relying on external teacher models. At inference time, hints are **not used**, meaning the final model operates normally without additional inputs.

Thesis Relevance

Small Language Models are the Future of Agentic AI (2506.02153v2)

- The authors explicitly argue that SLMs are sufficiently powerful, inherently more suitable, and necessarily more economical for agentic systems compared to LLMs. They argue that the dominance of LLMs in agent design is a misallocation of resources.
- **Decomposition and Narrow Scope:** The paper highlights that agentic applications expose only a very narrow subset of language model capabilities. Because complex goals are decomposed into modular sub-tasks, generalist LLMs are often overkill. This supports the system design where communication is restricted to the orchestrator and specialized sub-agents.
- Efficiency and Scaling: The paper provides a strong foundation for your efficiency research questions, noting that SLMs are 10-30x cheaper in latency, energy consumption, and

FLOPs.

- The authors outline an LLM-to-SLM conversion algorithm, emphasizing that organic data gathered from agentic interactions can be used to fine-tune SLMs overnight - role-specific fine-tuning.

In-the-Flow Agentic System Optimization for Effective Planning and Tool Use (2510.05592v1)

- Proof of "Small Beats Large": The authors instantiate their entire four-module system **(Planner, Executor, Verifier, Generator) using a 7B parameter model (Qwen2.5-7B). They prove that this SLM ensemble significantly outperforms the ~200B parameter GPT-4o across multiple domains**.
- **Pitfalls of Monolithic Models:** They explicitly argue that training a single, monolithic policy to interleave reasoning and tool calls scales poorly as horizons lengthen and tool diversity grows.

> In-the-Flow Agentic System Optimization for Effective Planning and Tool Use (2510.05592v1), p.10
>
> > However, this monolithic approach scales poorly as task complexity and planning horizons grow. The central challenge is long-horizon credit assignment; attributing a final outcome to specific intermediate tool calls remains difficult, even with fine-grained, turnlevel rewards (Zeng et al., 2025a; Wang et al., 2025d). This difficulty leads to training instability and brittle inference-time generalization, manifesting as strategic deficiencies like tool overuse or “cognitive offloading” (Wang et al., 2025b; Qian et al., 2025b), suboptimal personalization (Cheng et al., 2025), and poor alignment with user preferences for tool invocation (Huang et al., 2025).

- **RL Fine-Tuning Approach (Flow-GRPO):** This paper introduces an in-the-flow reinforcement learning technique that **trains the planner on-policy during live tool interactions, rather than relying on offline supervised fine-tuning** (which they prove actually degrades performance)
- They utilize an evolving memory to pass information between **distinct agent roles,** which prevents context window bloat and keeps the system transparent.

Multi-Agent Collaboration Mechanisms A Survey of LLMs (2501.06322v1)

- Architectural Taxonomy: This paper gives the exact academic terminology to describe the system. Our single-orchestrator, isolated-sub-agent design is officially a **Centralized Structure (Orchestrator as the hub) utilizing a Role-based Strategy (specialized sub-agents) with a Cooperative collaboration type.**

- The survey explicitly highlights that a single agent's failure (like a hallucination) can spread and be reinforced by other agents, leading to cascading effects. **This provides a strong theoretical motivation for the plan to investigate the robustness and failure modes of multi-agent systems compared to monolithic models.**
- **The "Orchestrator vs. Sub-agent" Dynamic:** The survey notes that centralized serving agents have the heavy objective of managing and controlling interactions. This perfectly aligns with our empirical finding that system performance is driven primarily by orchestrator capacity rather than sub-agent capacity.
- **NOTE**: implement a continuous external memory (a shared "mind map" for all sub-agents) → directly addresses the survey's point that interactions enable agents to "connect, negotiate, make decisions, plan, and act jointly". Implementing an Actor-Evaluator feedback loop within your orchestrator setup could also be a good architectural addition to test.

ToolOrchestra Elevating Intelligence via Efficient Model and Tool Orchestration (2511.21689v1)

- **Efficiency and scaling laws:** This research provides strong validation for the thesis's core hypothesis regarding resource constraints. It quantitatively demonstrates that a specialized 8B parameter orchestration model can consistently surpass a monolithic giant (e.g., GPT-5) on complex multi-step reasoning benchmarks (e.g. τ^2-Bench and FRAMES), achieving superior accuracy while consuming only about 30% of the cost.
- **Failure modes of monolithic models:** The study directly addresses the thesis objective of comparing the robustness of multi-agent systems to large single models. It reveals a critical failure mode: when standard monolithic LLMs are tasked with orchestration via simple prompting, they exhibit brittle decision-making, fail to optimize for efficiency, and suffer from "self-enhancement bias" (disproportionately invoking their own variants or blindly defaulting to the most expensive tools).
- The authors' use of multi-objective reinforcement learning aligns with the thesis's planned investigation into agent fine-tuning approaches. By incorporating efficiency-aware rewards (penalizing high cost and latency), the paper proves that targeted adaptation can actively train an SLM to be a vastly superior and more deliberate resource manager than a generalist LLM → **ad. post training and targeted adaptation**

Distilling LLM Agent into Small Models with Retrieval and Code Tools (2505.17612v2)

The paper introduces Agent Distillation, shifting the focus from distilling static Chain-of-Thought (CoT) reasoning to distilling interactive **"reason-act-observe" trajectories.** This teaches small language models (sLMs) how to actively use tools instead of trying to force them to memorize facts or perform complex math internally.

- The "How-To" for Small Agents: To make small models reliable, the authors introduce two key mechanisms: **First-Thought Prefix (FTP)** to guarantee the teacher model provides high-quality training data. **Self-Consistent Action Generation (SAG)** to prevent small models from crashing the system with bad code or tool calls at test time.
- Central orchestrator delegates to specialized sub-agents → this paper proves that those sub-agents can be extremely small (0.5B to 3B parameters) and still perform at the level of much larger models if they are trained via Agent Distillation. **It also provides a direct solution to the "failure modes" aspect by using SAG to filter out bad tool calls before they propagate back to your orchestrator.**
- **Tool-Augmented Sub-Agents:** exploring how augmenting agents with external tools (web search, code environments) influences performance. It proves that delegating knowledge retrieval and mathematical computation to tools allows small agents to circumvent their inherent parameter limitations.
- **Failure Modes and Robustness:** A core objective of the thesis is identifying the failure modes of MAS. This paper highlights a massive failure mode for small agents: generating invalid, misformatted, or unexecutable tool calls. The introduction of SAG provides a direct architectural solution for the thesis to consider when building robust execution pipelines.
- **Efficiency and Scaling Laws:** The paper directly addresses the efficiency trade-offs of agentic systems. A key finding is that distilled small agents do not necessarily generate significantly more tokens than CoT models. While they use more tokens for factual retrieval, they actually generate fewer tokens for math reasoning by writing concise code loops to handle repetitive calculations, optimizing the overall latency and compute.

Multi-modal Agent Tuning Building a VLM-Driven Agent for Efficient Tool Usage (2412.15606v2)

The work is relevant to the thesis in several ways:

1. It demonstrates that agent capabilities can be significantly improved through specialized training on tool-usage trajectories, rather than relying solely on prompt engineering or model scale. This aligns with the thesis goal of studying how architectural and training choices influence agentic performance.
2. **The paper shows that relatively small models (7–8B parameters) can become effective agent controllers when properly tuned, supporting the hypothesis that smaller models may perform competitively in well-designed agentic systems.**
3. The introduction of the MM-Traj dataset highlights the importance of high-quality trajectory data for training agents, suggesting a potential avenue for improving the performance of small-model agents in multi-agent systems.

Finally, the paper reinforces the importance of tool-usage reasoning and trajectory supervision, which directly relates to the thesis objective of studying how agents interact with external tools

and coordinate reasoning across multiple steps.

They use the following models: Qwen2-VL-7B, MiniCPM-V-8.5B
After tuning they become competitive agents. This supports the thesis hypothesis that: well-organized small models can rival larger systems.

Self-Consistency Improves Chain of Thought Reasoning in Language Models (2203.11171v4)

Self-Consistency Improves Chain of Thought Reasoning in Language Models (2203.11171v4), p.9

> We introduced a simple yet effective method called self-consistency, and observed that it significantly improves accuracy in a range of arithmetic and commonsense reasoning tasks, across four large language models with varying scales. Beyond accuracy gains, self-consistency is also useful for collecting rationales when performing reasoning tasks with language models, and for providing uncertainty estimates and improved calibration of language model outputs.
> **High level overview**

The proposed idea is relevant for research on small agents and modular reasoning systems because it demonstrates that performance improvements can be obtained through reasoning diversity rather than model scaling. Instead of increasing model size or training additional models, the method creates a self-ensemble within a single model by sampling multiple reasoning trajectories and aggregating their outputs. This insight motivates later work exploring whether multiple smaller models or agents can collaboratively approximate the reasoning ability of larger models, which aligns with research directions in multi-agent LLM systems and small-agent architectures. (see **SAG in Distilling paper**).

The main limitation of this approach is increased computational cost, since multiple generations must be produced for each query.

Self-Hinting Language Models Enhance Reinforcement Learning (2602.03143v1)

This paper shows that **structured intermediate guidance (hints)** can improve reasoning performance during reinforcement learning training, especially in sparse reward settings. The separation between **hint generation and solution generation** also resembles role decomposition seen in multi-agent reasoning systems. Additionally, it demonstrates that **smaller models can improve reasoning without increasing model size**, which is relevant for research on small language models and modular reasoning pipelines.

NOTE/REMINDER: GRPO learns by comparing multiple generated answers for the same prompt and pushing the model toward the answers that are better than the group average.

Self-Hinting Language Models Enhance Reinforcement Learning (2602.03143v1), p.2

> We propose SAGE (Self-hint Aligned GRPO with Privileged Supervision), a complementary approach based on privileged hinting. During training, we provide an additional hint h, a lossy compression of a reference solution τ ⋆, and roll out from the hint-conditioned policy πθ (· | x, h). Hints only reshape the rollout distribution to increase the probability of observing mixed outcomes within a finite group.

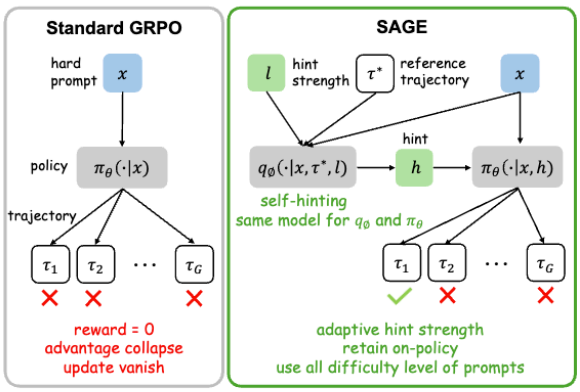

Figure 1. An overview of our proposed method, SAGE. When an LLM can't sample any correct trajectory for a prompt, the LLM **self-generates** hint from the reference solution of the prompt. The hint is then used together with the difficult prompt as input to the LLM, avoiding advantage collapse and ensuring the sampling of correct trajectories to update the policy model.

Self-Hinting Language Models Enhance Reinforcement Learning (2602.03143v1), p.3

> Summary. Sparse rewards can cause GRPO to stall because, for many prompts x, finite groups contain no positive samples and advantages collapse. Privileged hinting fixes this by changing the rollout distribution for such prompts while keeping the reward unchanged. A policy-dependent scheduler activates hints only when groups collapse, yielding an automatic curriculum. Training remains on-policy since rollouts are drawn from πθ (· | x, h). Deployment uses ℓ = 0 and requires no hints or privileged information.

- They used https://github.com/verl-project/verl for training and vLLM for sampling → consider verl!
- **consider SAGE-Light for less computational overhead**

🛠️ Example RQs (TBC, WIP)

Main Research Question (MRQ)

Can a centralized multi-agent system composed of Small Language Models (SLMs) match or exceed the performance of monolithic Large Language Models (LLMs) on complex reasoning and tool-use tasks while maintaining strict resource efficiency?

Logical Flow: This overarching question drives the entire thesis. To answer it, you must first define the architecture and test its performance (SQ1), measure its cost-effectiveness (SQ2), analyze where it breaks down (SQ3), and finally explore how to improve it through targeted adaptation (SQ4).

Sub-Questions (SQs) and Literature Connections

SQ1: Architecture and Performance *How does a centralized, role-based orchestration architecture utilizing specialized SLM sub-agents compare to monolithic LLMs in solving multi-step, tool-intensive reasoning tasks?*

- **Connection to Literature:** This directly tests the "orchestration paradigm" proposed in *ToolOrchestra* (Su et al., 2025) and the *AGENTFLOW* framework (Li et al., 2025), both of which argue that decoupling planning from execution yields superior reasoning. The terminology for the system design (Centralized Structure, Role-based Protocols) is grounded in the *Survey of LLMs* (Tran et al., 2025), while the hypothesis that SLMs are sufficient for these scoped execution roles is supported by *Small Language Models are the Future of Agentic AI* (Belcak et al., 2025).

SQ2: Efficiency and Scaling Trade-offs *What are the specific trade-offs between system performance and computational efficiency (measured by inference cost, latency, and memory usage) when distributing tasks across an SLM ensemble versus a single monolithic model?*

- **Connection to Literature:** *Small Language Models are the Future of Agentic AI* establishes that SLMs are mathematically 10-30x cheaper to serve than LLMs. *ToolOrchestra* provides the empirical methodology for this question, demonstrating how to measure the "cost vs. performance" frontier and proving that an 8B model can achieve frontier-level accuracy at 30% of the cost. *MaAS* (Zhang et al., 2025) also connects here by proving that complex, token-heavy systems are largely unnecessary for simpler tasks.

SQ3: Robustness and Failure Modes *What are the distinct failure modes of orchestrator-led SLM ensembles compared to monolithic LLMs, particularly concerning error propagation, tool-selection biases, and hallucination cascading?*

- **Connection to Literature:** *ToolOrchestra* provides the foundational motivation for this question by identifying that monolithic LLMs fail at orchestration due to "self-enhancement bias" (e.g., GPT-5 defaulting to GPT-5-mini regardless of task needs). Conversely, the *Survey of LLMs* (Tran et al., 2025) highlights the specific risk factor of multi-agent systems: a single agent's failure can spread and be reinforced by other agents (cascading hallucinations). *AGENTFLOW* further notes that monolithic models suffer from brittle generalization as task horizons lengthen.

SQ4: Targeted Adaptation and Fine-Tuning *How does targeted adaptation—specifically multi-objective, in-the-flow reinforcement learning applied to the SLM orchestrator—impact the system's ability to optimize tool usage, adapt to resource constraints, and improve overall collaborative performance?*

- **Connection to Literature:** *ToolOrchestra* demonstrates that an orchestrator must be explicitly trained (using techniques like Group Relative Policy Optimization) with efficiency-aware rewards to function correctly, rather than relying on static prompting. *AGENTFLOW* proves that offline Supervised Fine-Tuning (SFT) actually collapses multi-agent performance, necessitating "in-the-flow" RL to solve long-horizon credit assignment. *Small Language Models are the Future of Agentic AI* supports the feasibility of this by noting that SLM fine-tuning can be done overnight on consumer hardware.